

DOCUMENTAÇÃO TÉCNICA

WAREHOUSEFLOW

SPRINTS 01 — 04

Planejamento • Desenvolvimento • Testes • Consolidação

INFORMAÇÕES DO PROJETO

Projeto:	WarehouseFlow — Sistema de Gestão de Armazém e Estoque
Grupo:	26
Turma:	8MB
Disciplina:	Fábrica de Software
Entrega:	Sprints 01 a 04

EQUIPE

Gabriel George de Araújo Figueiredo	01605236
João Pedro Silva de Araujo	01606470
Sérgio José de Araújo Júnior	01590694
Thiago de Moraes Gonçalves	01609695
Guilherme Branco Ferrario	01596391

SPRINT 01 — PLANEJAMENTO DO PROJETO

WarehouseFlow

Nome inicial do projeto: WarehouseFlow — Sistema de Gestão de Armazém e Estoque.
Turma: 8MB

2. Escolha do tema

O projeto será desenvolvido no contexto de logística e gestão de armazéns, mais especificamente na área de controle de estoque (Warehouse Management). Esse contexto envolve o recebimento, armazenamento, movimentação, contagem e expedição de produtos dentro de um centro de distribuição ou depósito, sendo um domínio rico em processos, regras de negócio e oportunidades de automação.

3. Definição do problema

3.1 Qual problema a equipe pretende resolver?

Muitas pequenas e médias empresas ainda controlam seus estoques por meio de planilhas eletrônicas ou anotações manuais, o que gera divergências entre o estoque físico e o estoque registrado, dificuldade em localizar produtos dentro do armazém, falta de visibilidade sobre produtos com estoque baixo ou próximos da validade, e retrabalho na hora de gerar relatórios gerenciais. O WarehouseFlow pretende resolver esse problema oferecendo um sistema centralizado, que permita cadastrar produtos e localizações, registrar entradas, saídas e movimentações internas, e acompanhar o saldo de estoque em tempo real.

3.2 Por que esse problema é relevante?

O controle inadequado de estoque impacta diretamente o resultado financeiro das empresas, gerando perdas por produtos vencidos ou extraviados, rupturas de estoque que afetam vendas, e retrabalho operacional. Além disso, é um problema recorrente em pequenas e médias empresas, que muitas vezes não têm orçamento para adquirir sistemas de gestão robustos, tornando relevante o desenvolvimento de uma solução mais enxuta e acessível, focada especificamente na gestão de armazém.

4. Objetivos do sistema

4.1 Objetivo geral

Desenvolver um sistema para gestão de estoque e armazém que permita controlar, de forma centralizada e confiável, o cadastro de produtos, as movimentações de entrada, saída e transferência, e a geração de relatórios gerenciais que apoiem a tomada de decisão.

4.2 Principais resultados que a solução pretende alcançar

- Reduzir divergências entre o estoque físico e o estoque registrado no sistema.
- Aumentar a agilidade na localização de produtos dentro do armazém.
- Diminuir perdas por produtos vencidos ou parados em estoque.
- Fornecer informações gerenciais confiáveis para apoiar decisões de compra e venda.
- Padronizar os processos de recebimento, armazenagem e expedição de mercadorias.

5. Público-alvo

O sistema é voltado principalmente para pequenas e médias empresas que possuem um armazém, depósito ou centro de distribuição próprio, como comércios, indústrias de pequeno porte e distribuidoras. Dentro dessas empresas, os principais usuários do sistema serão:

- Operadores de estoque — responsáveis pelo recebimento, armazenagem, movimentação e expedição de produtos no dia a dia.
- Conferentes — responsáveis pela contagem física e conferência do inventário.
- Administradores/gestores — responsáveis por cadastros gerais, definição de permissões e acompanhamento de relatórios gerenciais.

6. Requisitos Funcionais

A tabela a seguir apresenta uma primeira relação das principais funcionalidades previstas para o sistema, que poderão ser detalhadas e ajustadas nas próximas Sprints.

Código	Descrição
RF01	Cadastrar, editar, consultar e inativar produtos (SKU, descrição, categoria, unidade de medida, código de barras).
RF02	Cadastrar e gerenciar localizações físicas do armazém (corredor, prateleira, nível, posição).
RF03	Registrar entradas de mercadorias (recebimento) vinculadas a fornecedor e nota/pedido de compra.
RF04	Registrar saídas de mercadorias (expedição) vinculadas a pedidos de venda ou requisições internas.
RF05	Movimentar estoque entre localizações dentro do armazém (transferência interna).
RF06	Controlar saldo de estoque em tempo real por produto e por localização.
RF07	Gerar alertas automáticos de estoque mínimo/ruptura e de produtos próximos da validade.
RF08	Realizar contagem cíclica/inventário rotativo e apontar divergências entre saldo físico e sistêmico.
RF09	Gerar relatórios gerenciais (giro de estoque, acurácia de inventário, produtos parados, curva ABC).
RF10	Emitir etiquetas e documentos de apoio (romaneio, etiqueta de localização, etiqueta de produto).
RF11	Controlar perfis de acesso (administrador, operador de estoque, conferente) com permissões distintas.
RF12	Registrar histórico (log) de todas as movimentações de estoque, com usuário e data/hora.

7. Requisitos Não Funcionais

Além das funcionalidades, o sistema deverá atender às seguintes características técnicas e de qualidade:

Código	Categoria	Descrição
RNF01	Usabilidade	A interface deve ser simples e objetiva, priorizando o uso rápido por operadores de armazém, inclusive em dispositivos móveis/coletores.
RNF02	Desempenho	Consultas de saldo de estoque devem responder em até 2 segundos, mesmo com grande volume de itens cadastrados.
RNF03	Segurança	Autenticação obrigatória por usuário e senha, com controle de perfis de acesso e criptografia de senhas.
RNF04	Confiabilidade	O sistema deve garantir a integridade do saldo de estoque mesmo em caso de falhas, evitando movimentações duplicadas.
RNF05	Disponibilidade	O sistema deve estar disponível para uso durante o horário de operação do armazém, com meta de disponibilidade de 99%.
RNF06	Portabilidade	A aplicação deve ser acessível via navegador web, sem necessidade de instalação local nas estações de trabalho.
RNF07	Manutenibilidade	O código deve seguir padrões de arquitetura em camadas, facilitando manutenção e evolução por diferentes desenvolvedores.
RNF08	Escalabilidade	O sistema deve suportar o crescimento do volume de produtos e movimentações sem perda relevante de desempenho.

8. Casos de Uso

A seguir são apresentados os principais casos de uso identificados nesta etapa inicial do planejamento.

Código	Caso de uso	Ator principal	Descrição resumida
UC01	Registrar entrada de mercadoria	Operador de estoque	Dar entrada de produtos recebidos de fornecedores, atualizando o saldo e a localização no armazém.
UC02	Registrar saída de mercadoria	Operador de estoque	Baixar produtos do estoque referentes a pedidos de venda ou requisições internas.
UC03	Movimentar produto entre localizações	Operador de estoque	Transferir um produto de uma posição para outra dentro do próprio armazém.
UC04	Consultar saldo de estoque	Operador de estoque / Administrador	Verificar a quantidade disponível de um produto, por localização ou de forma consolidada.
UC05	Realizar contagem de inventário	Conferente	Conferir fisicamente os produtos e registrar eventuais divergências com o sistema.
UC06	Cadastrar produto	Administrador	Incluir um novo produto no catálogo do sistema, com seus respectivos atributos.
UC07	Gerar relatórios gerenciais	Administrador	Emitir relatórios de desempenho do armazém, como giro de estoque e acurácia de inventário.
UC08	Gerenciar usuários e permissões	Administrador	Cadastrar usuários do sistema e definir seus perfis de acesso.

9. Product Backlog

O Product Backlog inicial organiza, em ordem de prioridade, os principais itens previstos para o desenvolvimento do sistema. Esta lista será refinada e detalhada em backlog de sprint (Sprint Backlog) ao longo do semestre.

#	Item do backlog	Prioridade	Requisito relacionado
1	Cadastro de produtos	Alta	RF01
2	Cadastro de localizações do armazém	Alta	RF02
3	Registro de entrada de mercadorias	Alta	RF03
4	Registro de saída de mercadorias	Alta	RF04
5	Controle de saldo de estoque em tempo real	Alta	RF06
6	Movimentação interna entre localizações	Média	RF05
7	Autenticação e controle de perfis de acesso	Alta	RF11
8	Alertas de estoque mínimo e validade	Média	RF07
9	Módulo de contagem/inventário rotativo	Média	RF08
10	Relatórios gerenciais (giro, curva ABC, acurácia)	Média	RF09
11	Emissão de etiquetas e romaneios	Baixa	RF10
12	Histórico (log) de movimentações	Média	RF12

10. Cronograma inicial

O cronograma abaixo apresenta uma organização preliminar das etapas previstas ao longo do semestre, que poderá ser ajustada conforme o andamento do projeto e as orientações recebidas.

Etapas	Principais atividades previstas
Sprint 01	Planejamento do projeto: tema, problema, objetivos, requisitos, backlog inicial e criação do repositório.
Sprint 02	Modelagem: diagramas de casos de uso detalhados, modelo de dados/ER e protótipo de telas (wireframes).
Sprint 03	Desenvolvimento do módulo de cadastros (produtos, localizações, usuários) e autenticação.
Sprint 04	Desenvolvimento das movimentações de estoque: entrada, saída e transferência interna.
Sprint 05	Desenvolvimento de alertas, contagem de inventário e ajustes de estoque.
Sprint 06	Desenvolvimento dos relatórios gerenciais e refinamento da interface.
Sprint 07	Testes integrados, correção de bugs e ajustes de desempenho.
Sprint 08	Entrega final, documentação do sistema e apresentação do projeto.

11. Repositório GitHub

O repositório da equipe já está criado e disponível no seguinte endereço:

<https://github.com/thiagodemorais87/WarehouseFlow>

Sumário

1. Introdução e visão da solução
2. Arquitetura do sistema
3. Diagrama de classes
4. Modelo Entidade-Relacionamento (MER)
5. Modelo relacional
6. Protótipo das telas principais
7. Banco de dados
8. Projeto estruturado no GitHub
9. Considerações finais

1. Introdução e visão da solução

O WarehouseFlow é um sistema proposto para apoiar o gerenciamento de operações de armazéns e estoques, centralizando informações de produtos, estoque, posições, pedidos e tarefas operacionais. O diferencial da solução é a incorporação de um motor de otimização capaz de analisar os dados disponíveis e produzir recomendações para tornar a execução das atividades mais eficiente.

A proposta busca reduzir deslocamentos desnecessários, melhorar a organização e a priorização das tarefas e auxiliar os operadores na tomada de decisão. Entre as funcionalidades previstas estão o cadastro de produtos, controle de estoque, gerenciamento de posições, registro de pedidos, controle de tarefas, sugestão de posições, sequenciamento de separação e otimização de deslocamentos.

1.1 Escopo funcional

Módulo	Responsabilidades principais
Produtos	Cadastro, consulta, atualização e exclusão de produtos.
Estoque	Controle de quantidade, entradas, saídas e localização dos produtos.
Armazém	Cadastro de armazéns e posições e controle de ocupação.
Pedidos	Cadastro, itens, status e prioridade dos pedidos.
Tarefas	Separação, movimentação, distribuição, priorização e controle de status.
Otimização	Sugestão de posição, priorização, sequenciamento e redução de deslocamentos.

2. Arquitetura do Sistema

A arquitetura adota uma separação lógica entre apresentação, aplicação, persistência e processamento computacional. Essa organização permite que as regras de negócio sejam centralizadas no back-end e que o motor de otimização seja integrado ao fluxo operacional sem depender diretamente da interface.

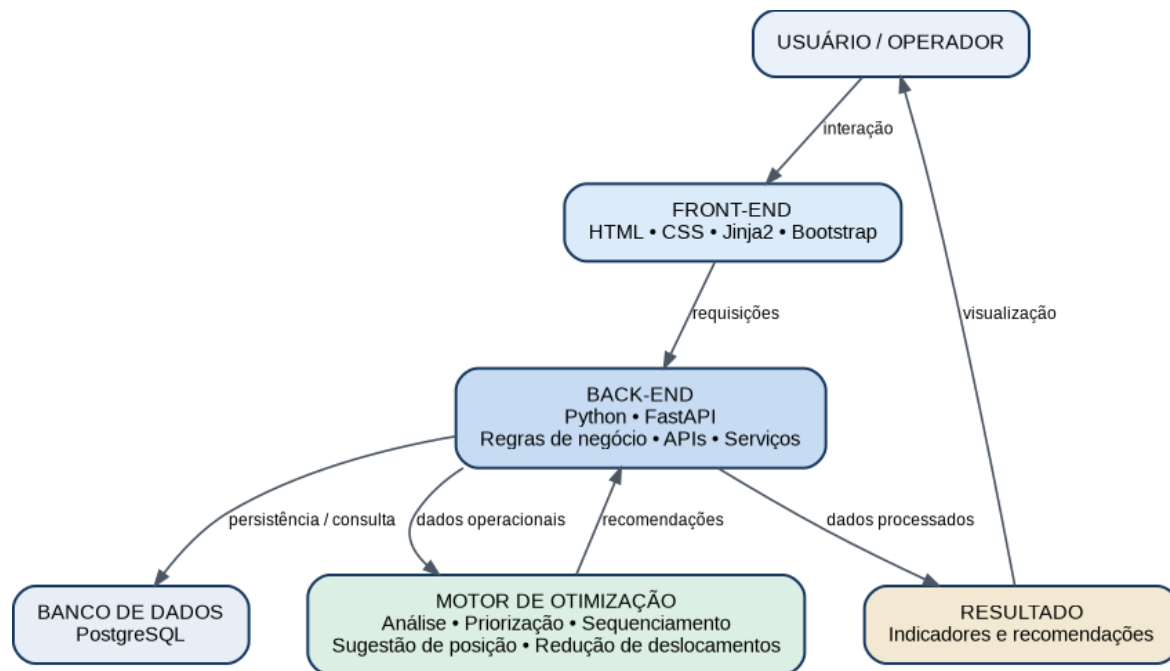


Figura 1 – Arquitetura lógica do WarehouseFlow.

2.1 Tecnologias

Camada	Tecnologias	Papel
Front-end	HTML, CSS, Jinja2, Bootstrap	Renderização das telas e interação com o usuário.
Back-end	Python, FastAPI, SQLAlchemy, Pydantic	APIs, regras de negócio, serviços, validação e acesso aos dados.
Dados	PostgreSQL	Persistência estruturada das informações do sistema.
Otimização	Motor de otimização	Análise dos dados operacionais e geração de recomendações.
Testes	Pytest	Automação de testes do código.
Infraestrutura	Docker, Docker Compose	Padronização e execução dos serviços.
Versionamento	Git, GitHub	Controle de versão e colaboração.

2.2 Integração com o componente computacional avançado

O motor de otimização recebe do back-end dados necessários para análise, como localização dos produtos, distâncias entre posições, prioridade dos pedidos, quantidade e disponibilidade de estoque, capacidade das posições e tarefas pendentes. A partir desses dados, o componente produz uma recomendação que retorna ao fluxo da aplicação e pode ser apresentada ao operador.

Fluxo lógico:

Pedidos pendentes → Produtos necessários → Localizações → Distâncias e prioridades → Algoritmo de otimização → Sequência/recomendação de tarefas

2.3 Requisitos não funcionais arquiteturais

- Separação de responsabilidades entre interface, aplicação, dados e otimização.
- Validação de dados na camada de aplicação.

- Persistência em banco relacional.
- Possibilidade de execução padronizada por containers.
- Versionamento e integração das alterações por Git/GitHub.

3. Diagrama de Classes

O diagrama de classes representa o domínio da solução e os principais objetos necessários para suportar os módulos descritos no projeto. As classes abaixo devem orientar a implementação do domínio e a persistência.

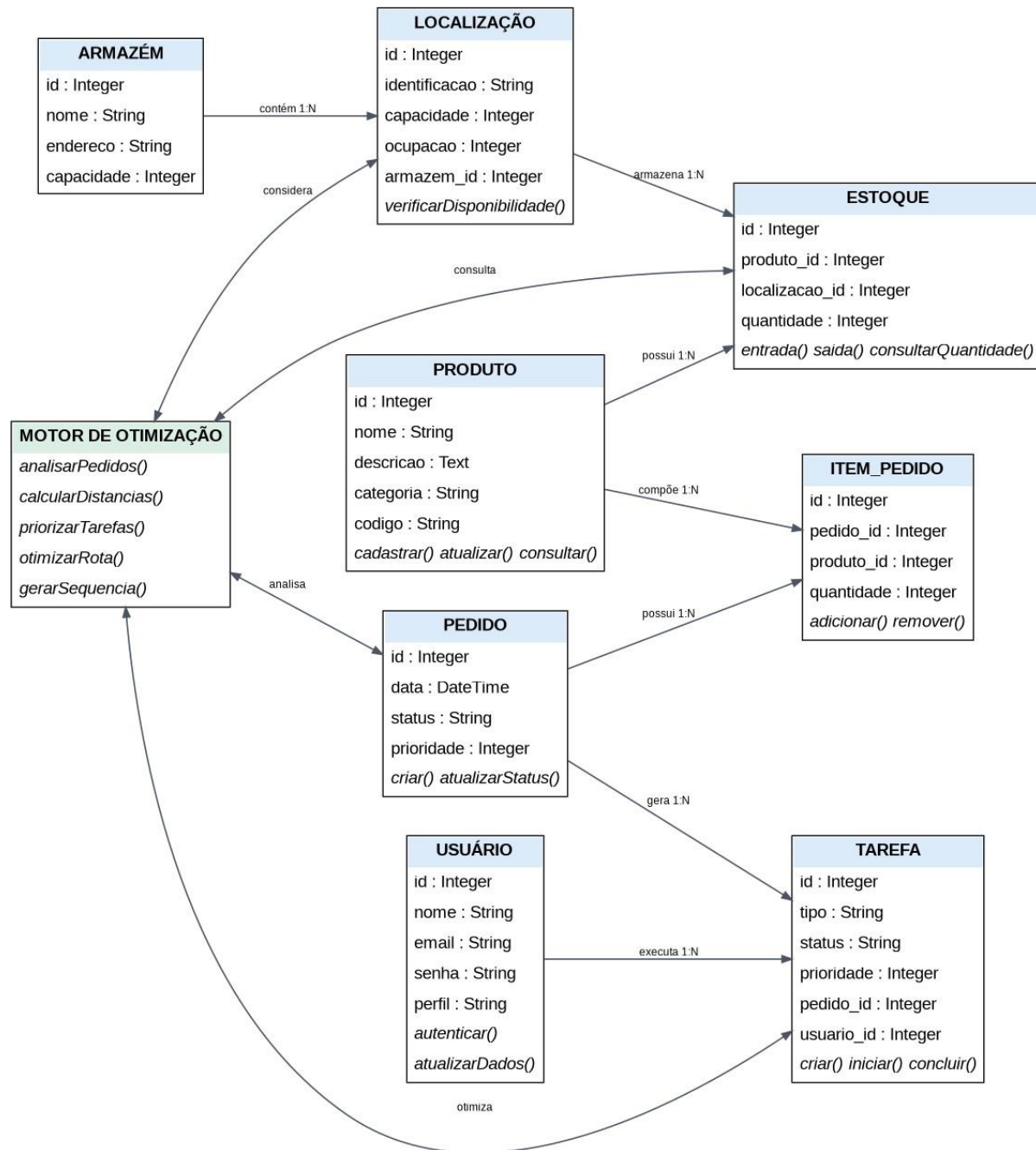


Figura 2 — Diagrama de classes conceitual do WarehouseFlow.

3.1 Responsabilidades das classes

Classe	Responsabilidade
Usuário	Representar o operador e seus dados de acesso/perfil.
Produto	Representar os itens administrados pelo armazém.
Armazém	Representar a unidade física e sua capacidade.
Localização	Representar uma posição física dentro do armazém.
Estoque	Relacionar produto, localização e quantidade disponível.
Pedido	Representar uma solicitação de produtos e sua prioridade/status.
ItemPedido	Representar cada produto e quantidade dentro de um pedido.
Tarefa	Representar uma atividade operacional derivada do fluxo de pedidos.
Motor de Otimização	Analisar dados operacionais e gerar recomendações para tarefas.

3.2 Relacionamentos principais

- Um armazém contém várias localizações (1:N).
- Uma localização pode possuir vários registros de estoque (1:N).
- Um produto pode possuir vários registros de estoque (1:N).
- Um pedido possui vários itens de pedido (1:N).
- Um produto pode aparecer em vários itens de pedido (1:N).
- Um pedido pode gerar várias tarefas (1:N).
- Um usuário pode executar várias tarefas (1:N).

4. Modelo Entidade-Relacionamento (MER)

O MER apresenta as entidades persistentes e os relacionamentos necessários para representar o domínio do WarehouseFlow. O modelo privilegia a separação entre cadastro, localização física, saldo de estoque, pedidos e execução operacional.

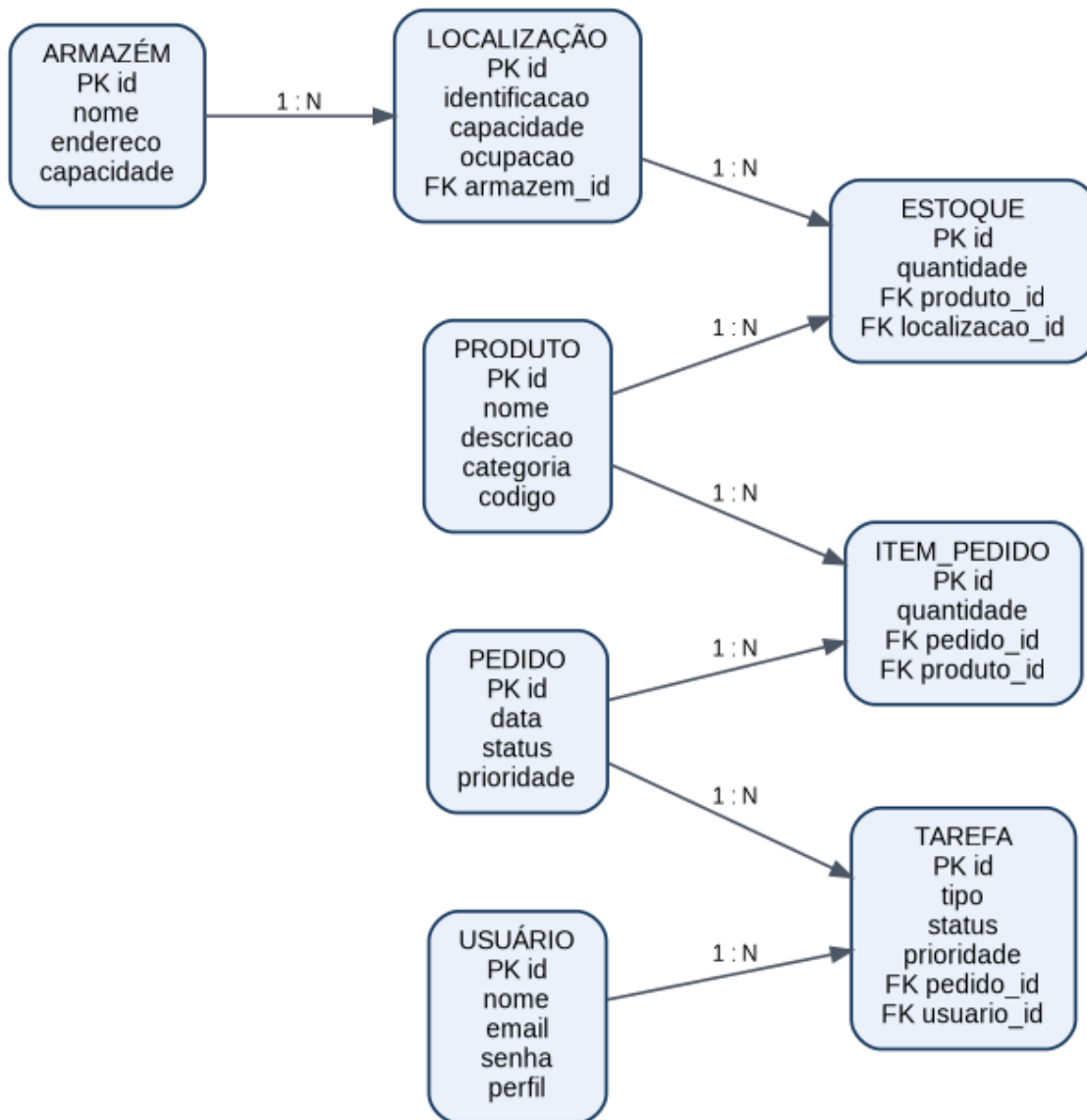


Figura 3 — Modelo Entidade-Relacionamento conceitual.

4.1 Entidades e atributos

Entidade	Atributos principais
USUÁRIO	id, nome, email, senha, perfil
ARMAZÉM	id, nome, endereço, capacidade
LOCALIZAÇÃO	id, identificação, capacidade, ocupação, armazém
PRODUTO	id, nome, descrição, categoria, código
ESTOQUE	id, produto, localização, quantidade
PEDIDO	id, data, status, prioridade
ITEM_PEDIDO	id, pedido, produto, quantidade
TAREFA	id, tipo, status, prioridade, pedido, usuário

5. Modelo Relacional

A transformação do modelo conceitual para o modelo relacional resulta nas tabelas abaixo. As chaves primárias identificam unicamente os registros, enquanto as chaves estrangeiras preservam os vínculos entre as entidades.

USUARIO

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
nome	VARCHAR	
email	VARCHAR	UNIQUE
senha	VARCHAR	
perfil	VARCHAR	

ARMAZEM

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
nome	VARCHAR	
endereco	VARCHAR	
capacidade	INTEGER	

LOCALIZACAO

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
identificacao	VARCHAR	
capacidade	INTEGER	
ocupacao	INTEGER	
armazem_id	INTEGER	FK → ARMAZEM.id

PRODUTO

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
nome	VARCHAR	
descricao	TEXT	
categoria	VARCHAR	
codigo	VARCHAR	UNIQUE

ESTOQUE

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
produto_id	INTEGER	FK → PRODUTO.id
localizacao_id	INTEGER	FK → LOCALIZACAO.id
quantidade	INTEGER	

PEDIDO

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
data	DATETIME	
status	VARCHAR	
prioridade	INTEGER	

ITEM_PEDIDO

Atributo	Tipo	Chave / restrição
----------	------	-------------------

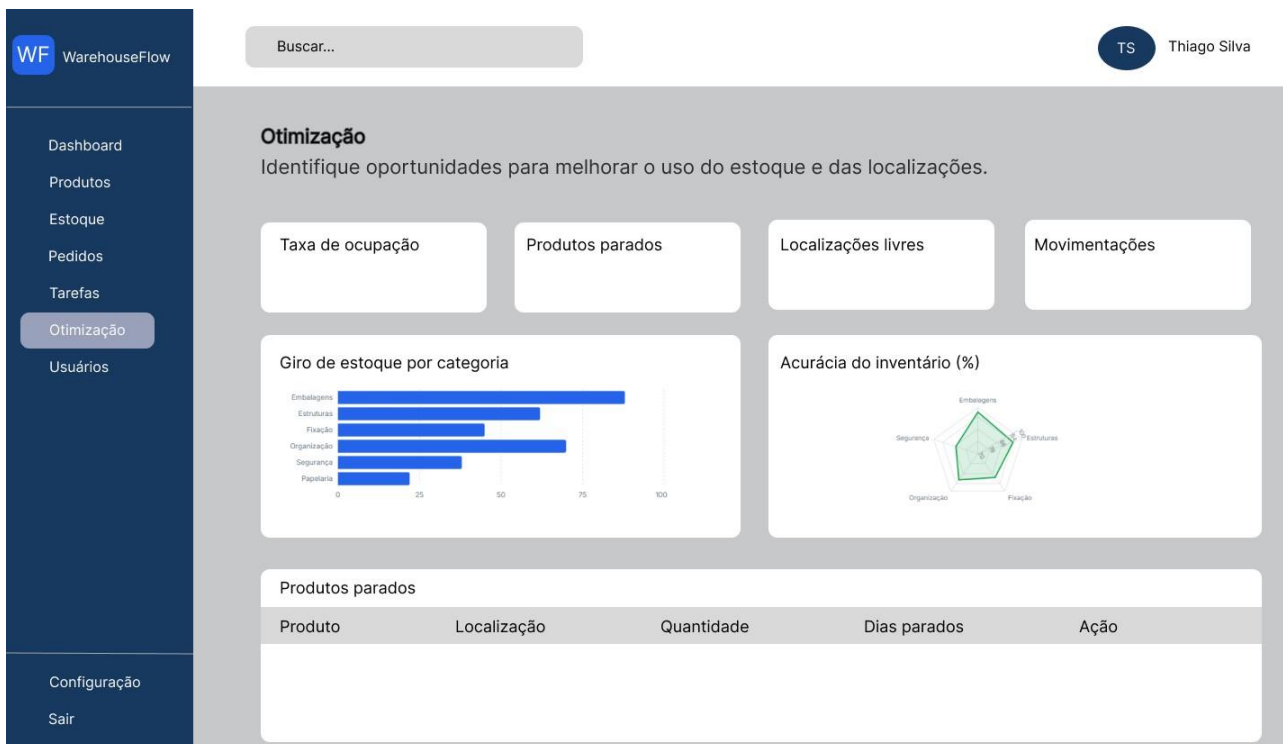
id	INTEGER	PK
pedido_id	INTEGER	FK → PEDIDO.id
produto_id	INTEGER	FK → PRODUTO.id
quantidade	INTEGER	

TAREFA

Atributo	Tipo	Chave / restrição
id	INTEGER	PK
tipo	VARCHAR	
status	VARCHAR	
prioridade	INTEGER	
pedido_id	INTEGER	FK → PEDIDO.id
usuario_id	INTEGER	FK → USUARIO.id

6. Protótipo das Telas Principais

O protótipo visual está no Figma e representa a navegação por Dashboard, Produtos, Estoque, Pedidos, Tarefas, Otimização, Usuários, Configuração e Sair. Para esta versão, foram incorporados ao documento os prints disponibilizados pela equipe, mantendo o link do protótipo completo como referência.



WF WarehouseFlow

Dashboard

Produtos

Estoque

Pedidos

Tarefas

Otimização

Usuários

Configuração

Sair

Buscar...

TS Thiago Silva

Usuários

Gerencie os usuários e seus níveis de acesso.

Novo usuário

Total

Ativos

Administradores

Operadores

Buscar por nome ou e-mail...

Todos

Todos

Usuário	E-mail	Perfil	Status	Ultimo acesso	Ações
---------	--------	--------	--------	---------------	-------

 WarehouseFlow

Bem-Vindo ao WarehouseFlow

Gerencie seu armazenamento de forma simples e eficiente.

E-mail:

Senha:

☐ Lembrar de mim

[Esqueci a senha](#)

Entrar

Grupo 26 • 8MB • Fábrica de Software

WF WarehouseFlow

Dashboard

Produtos

Estoque

Pedidos

Tarefas

Otimização

Usuários

Configuração

Sair

Buscar...

TS Thiago Silva

Ver estoque

Novo Produto

Dashboard

Visão geral do armazém

Total de produtos

Estoque total

Pedidos pendentes

Tarefas em andamento

Movimentações - ultimo mês

Estoque por categoria

Alertas

Movimentações recentes

WF WarehouseFlow

Dashboard

Produtos

Estoque

Pedidos

Tarefas

Otimização

Usuários

Configuração

Sair

Buscar...

TS Thiago Silva

Pedidos

Novo Pedido

Acompanhe recebimentos, expedições e pedidos internos.

todos

Entradas

Saídas

Pendentes

Concluídos

Buscar por número ou fornecedor/cliente...

pedido	Tipo	Fornecedor/cliente	itens	data	Status	ações
--------	------	--------------------	-------	------	--------	-------

WF WarehouseFlow

Dashboard

Produtos

Estoque

Pedidos

Tarefas

Otimização

Usuários

Configuração

Sair

Buscar...

TS Thiago Silva

Estoque

Acompanhe o saldo e a localização dos produtos

Estoque total

Estoque baixo

Sem estoque

Localizações ocupadas

Mapa do armazem

Normal Baixo Sem estoque Livre

	01	02	03	04	05	06
A	A01	A02	A03	A04	A05	A06
B	B01	B02	B03	B04	B05	B06
C	C01	C02	C03	C04	C05	C06
D	D01	D02	D03	D04	D05	D06

Buscar por produtos

Produto	Cód.	Qtd.	Local	Mín.	Status	Ação
---------	------	------	-------	------	--------	------

Grupo 26 • 8MB • Fábrica de Software



Link: <https://www.figma.com/design/uGceagehdqafcFckYSIWZY/Sem-t%C3%ADulo?node-id=5-569Ct=kWlwVzC0cuJRlaCp-1>

7. Banco de Dados criado

O banco de dados desta Sprint 02 não é apenas uma proposta conceitual. O repositório atual contém o schema e o seed utilizados para inicialização do PostgreSQL, além do Docker Compose que monta esses arquivos no container do banco

7.1 Artefatos do banco

backend/database/schema.sql	DDL das 10 tabelas, chaves primárias/estrangeiras, restrições e tabela optimization_results.
backend/database/seed.sql	Dados iniciais para roles, usuários, armazém, localizações, produtos, estoque, pedidos, tarefas e resultado de otimização.
docker-compose.yml	Serviço PostgreSQL 16, volume persistente e montagem automática de schema.sql + seed.sql.
SQLAlchemy / DATABASE_URL	Integração do back-end Python com PostgreSQL.

7.2 Evidência verificável no GitHub

Na versão atual do repositório, a existência do banco é verificável diretamente pelos arquivos publicados. O schema.sql define roles, users, products, warehouses, locations, stock, orders, order_items, tasks e optimization_results. O seed.sql insere dados iniciais nessas estruturas. O docker-compose.yml configura o PostgreSQL e monta os dois scripts para inicialização

Trecho representativo do schema.sql

```
CREATE TABLE roles (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(50) UNIQUE NOT NULL  
);  
  
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role_id INTEGER REFERENCES roles(id) ON DELETE SET NULL  
);  
  
CREATE TABLE optimization_results (  
  id SERIAL PRIMARY KEY,  
  task_id INTEGER REFERENCES tasks(id) ON DELETE CASCADE,  
  suggested_location_id INTEGER REFERENCES locations(id),  
  suggested_route JSONB,  
  score DECIMAL(10,4)  
);
```

Links de conferência:

<https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/database/schema.sql> •
<https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/database/seed.sql> •
<https://github.com/thiagodemorais87/WarehouseFlow/blob/main/docker-compose.yml>

8. Projeto Estruturado no GitHub

Repositório oficial: <https://github.com/thiagodemorais87/WarehouseFlow>

O repositório atual apresenta uma estrutura concreta, não apenas planejada. A raiz contém backend, database, docs, frontend, tests, Dockerfile, docker-compose.yml, requirements.txt, pytest.ini e README.md. Dentro de backend/database estão schema.sql e seed.sql; dentro de backend/optimization está o motor próprio de otimização.

Estrutura	Função
backend/app/	FastAPI, routers CRUD, schemas e serviços.
backend/database/	schema.sql e seed.sql.
backend/optimization/	Motor Python de otimização (Nearest Neighbor + 2-opt).
frontend/	Templates Jinja e arquivos estáticos.
docs/	Proposta, cronograma e documentação técnica do motor.
tests/	Testes automatizados.
docker-compose.yml	PostgreSQL + API em ambiente local.
pytest.ini / requirements.txt	Configuração e dependências dos testes.

8.1 Estratégia de versionamento

A implementação própria está em `backend/optimization/`, sem uso de OR-Tools ou outro solver externo. O pipeline utiliza Nearest Neighbor para construir uma rota gulosa e, em seguida, 2-opt para melhoria local. A distância é calculada pela métrica Manhattan.

Métrica	Significado
<code>distance_before</code>	Distância da rota original.
<code>nearest_neighbor_distance</code>	Distância após o Nearest Neighbor.
<code>two_opt_distance / distance_after</code>	Distância após o 2-opt.
<code>distance_reduction</code>	Ganho absoluto de distância: <code>before - after</code> .
<code>reduction_percent</code>	Ganho percentual em relação à rota original.
<code>execution_time_ms</code>	Tempo de execução do motor em milissegundos.

9. Considerações Finais

A revisão desta documentação corrige os pontos de coerência identificados na versão anterior. A identificação da equipe está corrigida, com Guilherme Branco Ferrario na matrícula 01596391; o modelo relacional foi alinhado ao schema atual; a seção de banco de dados passa a registrar evidências concretas do repositório; e o diferencial computacional deixou de ser descrito de forma genérica. A entrega também explicita o motor próprio de rota — Nearest Neighbor + 2-opt —, suas métricas `distance_before`, `reduction_percent` e `execution_time_ms`, além do endpoint `POST /optimization/route`. Dessa forma, arquitetura, banco, modelo relacional, GitHub e componente computacional são apresentados de maneira consistente com os artefatos atualmente publicados.

Referências

- GitHub — <https://github.com/thiagodemorais87/WarehouseFlow>
- Schema — <https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/database/schema.sql>
- Seed — <https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/database/seed.sql>
- Docker Compose — <https://github.com/thiagodemorais87/WarehouseFlow/blob/main/docker-compose.yml>
- Motor — <https://github.com/thiagodemorais87/WarehouseFlow/blob/main/backend/optimization/README.md>
- Protótipo — <https://www.figma.com/design/uGceagehdqafcFckYSIWZY/Sem-título?node-id=5-569>

1. Identificação e objetivo da Sprint 03

A Sprint 03 representa a etapa de desenvolvimento do WarehouseFlow voltada à implementação das funcionalidades de autenticação, cadastro e controle de perfis, além da integração da aplicação com o banco de dados e da disponibilização do CRUD principal. Nesta etapa, a estrutura planejada nas Sprints anteriores passa a ser demonstrada por meio de funcionalidades executáveis e evidências de uso.

1.1 Escopo desta entrega

- Autenticação de usuários por meio de login.
- Cadastro de usuários e gerenciamento de perfis de acesso.
- Cadastro e manutenção dos registros principais do sistema, com operações de CRUD.
- Persistência dos dados em PostgreSQL.
- Execução local do sistema por meio da estrutura definida no projeto.
- Organização e atualização do código no repositório GitHub.

1.2 Relação com o planejamento anterior

Nas Sprints 01 e 02 foram definidos o problema, objetivos, requisitos, arquitetura, modelos de dados, protótipo e organização do projeto. A Sprint 03 dá continuidade a esse planejamento, materializando principalmente os requisitos relacionados a cadastro de produtos, autenticação e controle de perfis.

2. Breve descrição da estrutura implementada

O WarehouseFlow está organizado em módulos que separam a aplicação, a persistência, a interface, os testes e os artefatos de infraestrutura. A raiz do projeto contém os diretórios backend, database, docs, frontend e tests, além dos arquivos de configuração e execução.

- backend/ — APIs, regras de negócio, persistência e serviços.
- backend/database/ — schema e dados iniciais do PostgreSQL.
- backend/optimization/ — motor próprio de otimização.
- frontend/ — interface da aplicação.
- tests/ — testes automatizados.
- docs/ — documentação e artefatos.
- docker-compose.yml — orquestração dos serviços.
- requirements.txt — dependências Python.
- pytest.ini — configuração dos testes.

A implementação mantém a arquitetura apresentada nas Sprints anteriores, com Python/FastAPI no backend, PostgreSQL na persistência e interface web baseada em HTML/CSS/Jinja2/Bootstrap.

3. Evidências do banco de dados conectado

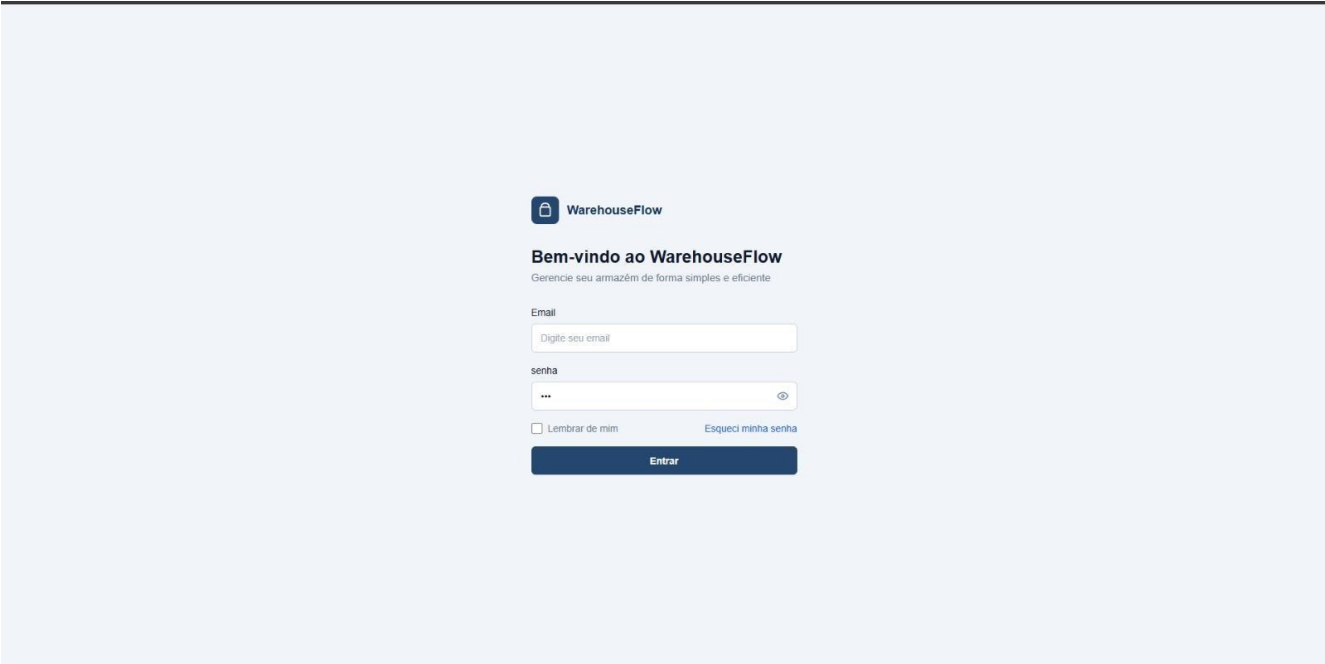
O sistema utiliza PostgreSQL como banco de dados relacional. A estrutura do projeto contém os artefatos necessários para criação e carga inicial do banco, e a aplicação utiliza a conexão configurada para persistir e consultar os dados dos módulos desenvolvidos.

.

A Sprint 03 contempla a autenticação dos usuários e o controle de acesso por perfil. O fluxo de autenticação identifica o usuário antes do acesso às funcionalidades protegidas, enquanto o cadastro e os perfis organizam os diferentes níveis de acesso previstos para o sistema.

4.1 Login

A tela de login é a porta de entrada do sistema. O usuário informa suas credenciais e, quando válidas, é direcionado para a aplicação. A autenticação deve impedir o acesso às áreas protegidas por usuários não autenticados.



The screenshot shows the WarehouseFlow login interface. At the top, there is a WarehouseFlow logo consisting of a blue square with a white padlock icon and the text 'WarehouseFlow'. Below the logo, the heading 'Bem-vindo ao WarehouseFlow' is displayed, followed by the tagline 'Gerencie seu armazém de forma simples e eficiente'. The login form includes an 'Email' field with the placeholder text 'Digite seu email', a 'senha' (password) field with three asterisks and a toggle icon, a 'Lembrar de mim' checkbox, a link for 'Esqueci minha senha', and a dark blue 'Entrar' button.

4.2 Cadastro de usuário

O cadastro permite registrar novos usuários para utilização do sistema. A implementação deve validar os dados informados e armazená-los de forma persistente, respeitando as regras de acesso definidas.

4.3 Controle de perfis

O controle de perfis associa usuários a funções/permisões do sistema. Essa separação permite que funcionalidades administrativas sejam disponibilizadas de acordo com o perfil do usuário autenticado.

5. Evidências do CRUD principal funcionando

O CRUD (Create, Read, Update e Delete) representa as operações de criação, consulta, alteração e exclusão de registros. Nesta Sprint, o CRUD principal deve ser demonstrado preferencialmente no módulo de Produtos, por ser uma das funcionalidades centrais do WarehouseFlow.

Create — Criar

Registro de um novo produto no sistema.

Read — Consultar

Visualização dos produtos cadastrados e seus dados.

Update — Atualizar

Alteração de informações de um produto existente.

Delete — Excluir

Remoção de um produto conforme as regras de negócio

6. Procedimento de execução local

O projeto pode ser executado localmente a partir da pasta raiz WarehouseFlow. Como a configuração do projeto utiliza Docker Compose, o procedimento recomendado é utilizar os arquivos de infraestrutura versionados no repositório. Caso a equipe utilize execução manual do Python, devem ser respeitadas as dependências e configurações definidas no projeto.

6.1 Pré-requisitos

- Python 3 instalado, quando utilizada a execução manual do back-end.
- Docker e Docker Compose instalados, quando utilizada a execução por containers.
- Git, caso seja necessário atualizar o código a partir do GitHub.

6.2 Execução por Docker Compose

Na pasta raiz do projeto, executar o Docker Compose conforme as instruções atuais do README. A execução deve iniciar os serviços necessários e disponibilizar a aplicação para acesso pelo navegador.

Comando de referência: docker compose up --build

Após a inicialização, acessar o endereço local informado pelo projeto/terminal e verificar o login e as funcionalidades desenvolvidas.

☐	▼	●	warehouseflow	-	-	-	5.38%	143.74MB / 11.1i	3.8%	16.4KB /				
☐		●	db	cf5e5665e6a3	postgres:1	5433:5432	3.22%	40.36MB / 3.7GB	1.07%	16.4KB /				
☐		●	api	09461b5d9c9f	warehouse	8000:8000	2.16%	95.74MB / 3.7GB	2.53%	0B / 0B				
☐		●	frontend	abae4118fef7	nginx:alpin	8080:80	0%	7.64MB / 3.7GB	0.2%	0B / 4.1i				

7. Link do repositório GitHub atualizado

Repositório oficial: <https://github.com/thiagodemorais87/WarehouseFlow>

8. Dificuldades encontradas

Durante a Sprint 03, as principais dificuldades estão relacionadas à integração entre as diferentes camadas da aplicação, configuração do ambiente local, conexão com o PostgreSQL, autenticação e organização dos perfis de acesso

9. Próximos passos

- Ampliar a cobertura de testes automatizados e testes integrados.
- Consolidar as movimentações de estoque e suas regras de negócio.
- Implementar/refinar alertas, inventário e ajustes de estoque conforme o backlog.
- Aprimorar relatórios gerenciais e indicadores operacionais.

- Realizar correções de bugs e ajustes de desempenho.
- Manter documentação, código e modelo de dados alinhados ao estado efetivamente implementado.

10. Considerações finais

A Sprint 03 representa a transição da etapa de modelagem para a comprovação prática das funcionalidades do WarehouseFlow. A implementação de autenticação, cadastro, controle de perfis, persistência e CRUD estabelece a base operacional necessária para as próximas etapas do projeto.

Antes da exportação final para PDF, os espaços reservados devem ser substituídos pelas evidências reais da execução. Recomenda-se que cada print seja acompanhado de uma legenda objetiva e mostre claramente a funcionalidade descrita na seção correspondente.

11. Referências

GitHub — WarehouseFlow: <https://github.com/thiagodemorais87/WarehouseFlow>

Figma — WarehouseFlow: <https://www.figma.com/design/uGceagehdqafcFckYSIWZY/Sem-título?node-id=5-569>

Documentação das Sprints 01 e 02 — material produzido pela equipe.

1. Objetivo do Sprint 04

O Sprint 04 tem como foco a consolidação do Módulo 1 — Produtos e Estoque (com posições), contemplando as operações de produtos, estoque e localizações/posições, além da validação da API por meio de cenários de teste.

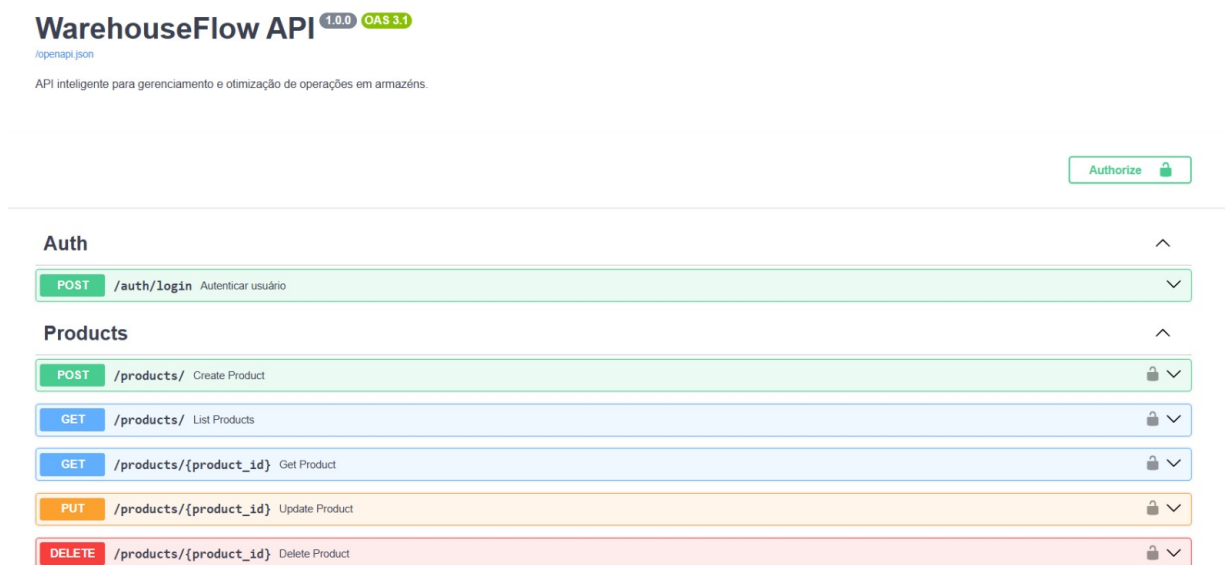
2. Escopo

- Cadastro, consulta, edição e exclusão de produtos.
- Criação/ajuste de estoque associado a produto e localização.
- Cadastro e listagem de localizações/posições.
- Validações de entrada, incluindo quantidade de estoque não negativa.
- Execução dos cenários pelo Postman contra a API local.

3. Integração e execução

Os cenários foram executados utilizando o Postman contra a API local do WarehouseFlow. As evidências apresentam os códigos HTTP e os corpos JSON retornados pela aplicação. A API conta ainda com documentação interativa (Swagger/OpenAPI), utilizada como referência para a execução dos testes.

API → Produtos → Estoque → Localizações/Posições



Evidência 01 — Documentação interativa da API (Swagger/OpenAPI) do WarehouseFlow, com os endpoints de autenticação (/auth/login) e de produtos (/products/).

4. Operações evidenciadas

Funcionalidade	Operação	Resultado
Produtos	POST /products/	201 Created
Produtos	PUT /products/{product_id}	200 OK
Produtos	DELETE /products/{product_id}	204 No Content

Funcionalidade	Operação	Resultado
Estoque	POST /stock/	201 Created
Localizações	POST /locations/	201 Created
Localizações	GET /locations/	200 OK
Validação	POST /stock/ com quantity = -10	422 Unprocessable Entity

5. Cenários de teste

ID	Cenário	Resultado esperado	Resultado observado
CT01	Cadastrar produto válido	Produto criado.	201 Created — Aprovado
CT02	Editar produto	Dados atualizados.	200 OK — Aprovado
CT03	Excluir produto	Produto removido.	204 No Content — Aprovado
CT04	Criar/ajustar estoque	Registro criado com produto, localização e quantidade.	201 Created — Aprovado
CT05	Criar localização	Localização vinculada ao armazém criada.	201 Created — Aprovado
CT06	Informar estoque negativo	Operação rejeitada pela validação.	422 Unprocessable Entity — Aprovado
CT07	Listar localizações	Localizações retornadas pela API.	200 OK — Aprovado

6. Evidências dos testes

As imagens abaixo foram fornecidas após a execução dos testes no Postman e registram os resultados observados.

Guilherme Ferrario's Workspace

WarehouseFlow - Sprint 4 > 02 - Produtos > Criar Produto

POST `{{base_url}} /products/` Send

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 4 Test Results 4/4 Visualize

201 Created - 56 ms - 287 B

```
{
  "sku": "PROD-SPRINT4-001",
  "name": "Produto Teste Sprint 4",
  "description": "Produto criado para testes automatizados",
  "weight": "1.60",
  "volume": "0.02",
  "id": 4
}
```

CT 01 — Cadastro de produto válido. Retorno 201 Created.

Guilherme Ferrario's Workspace

WarehouseFlow - Sprint 4 > 02 - Produtos > Editar Produto

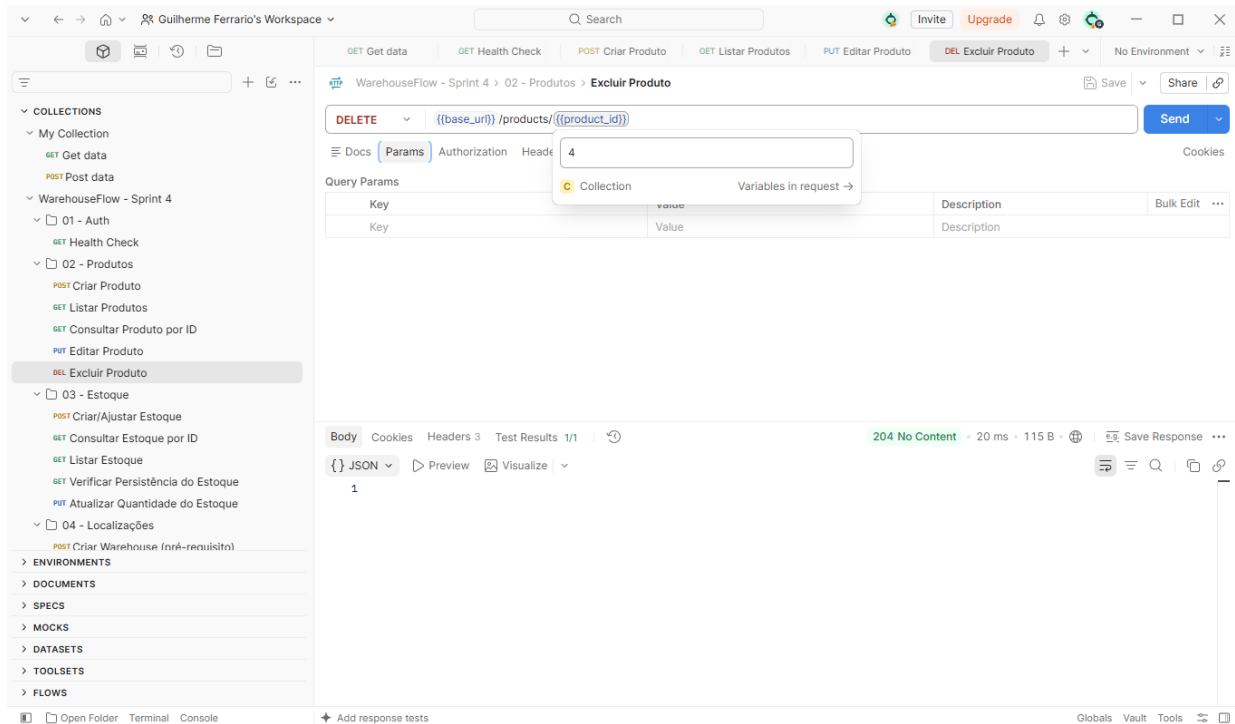
PUT `{{base_url}} /products/ {{product_id}}` Send

Body Cookies Headers 4 Test Results 3/3 Visualize

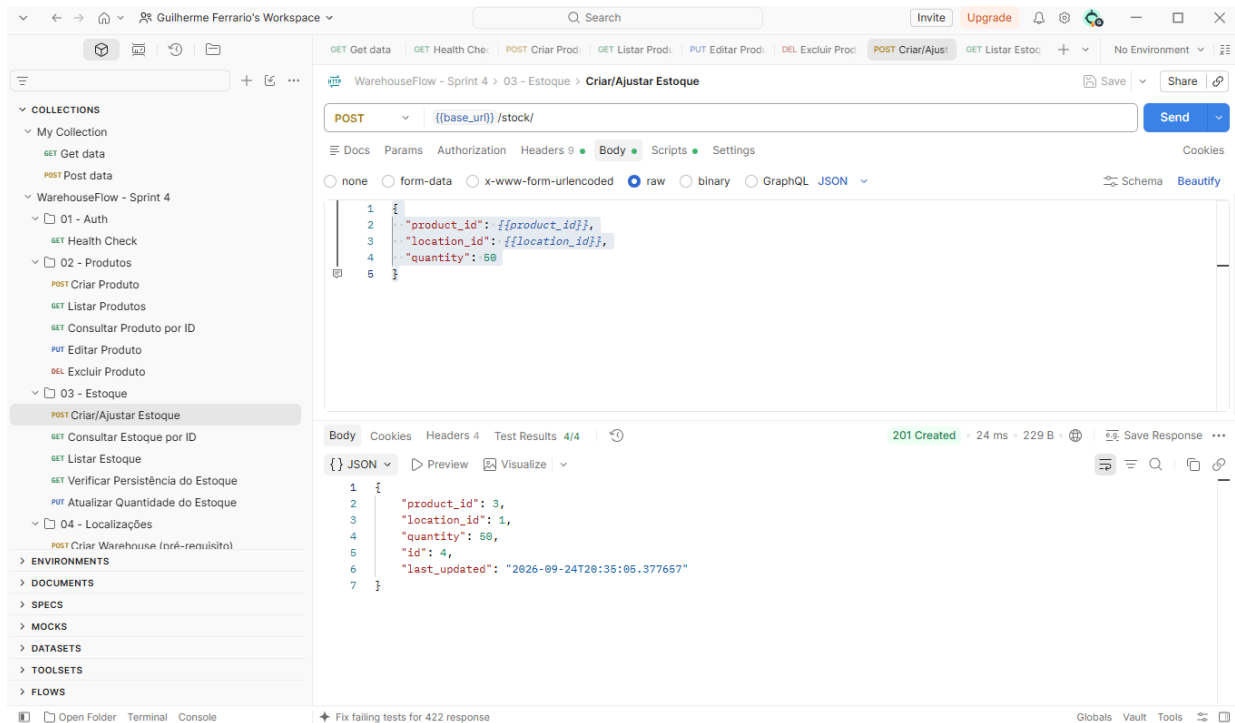
200 OK - 20 ms - 274 B

```
{
  "sku": "PROD-SPRINT4-001",
  "name": "Produto Teste Sprint 4 - Editado",
  "description": "Descrição atualizada",
  "weight": "2.00",
  "volume": "0.02",
  "id": 4
}
```

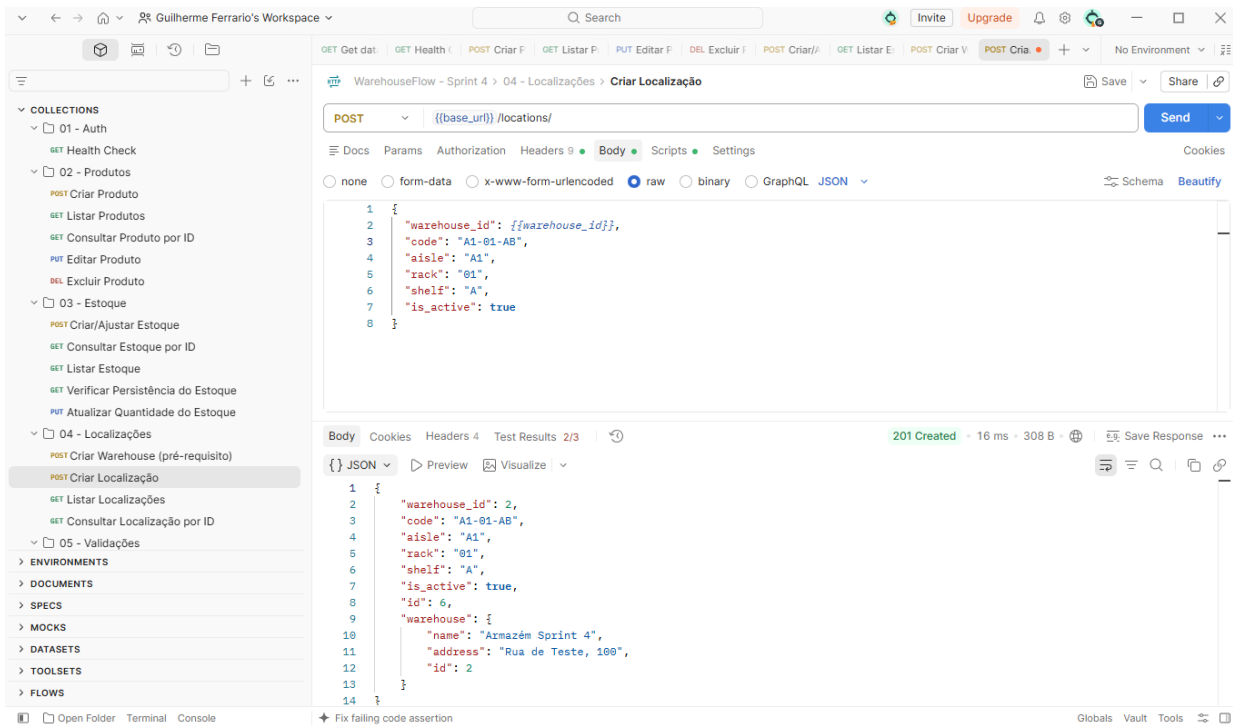
CT 02 — Edição de produto. Retorno 200 OK.



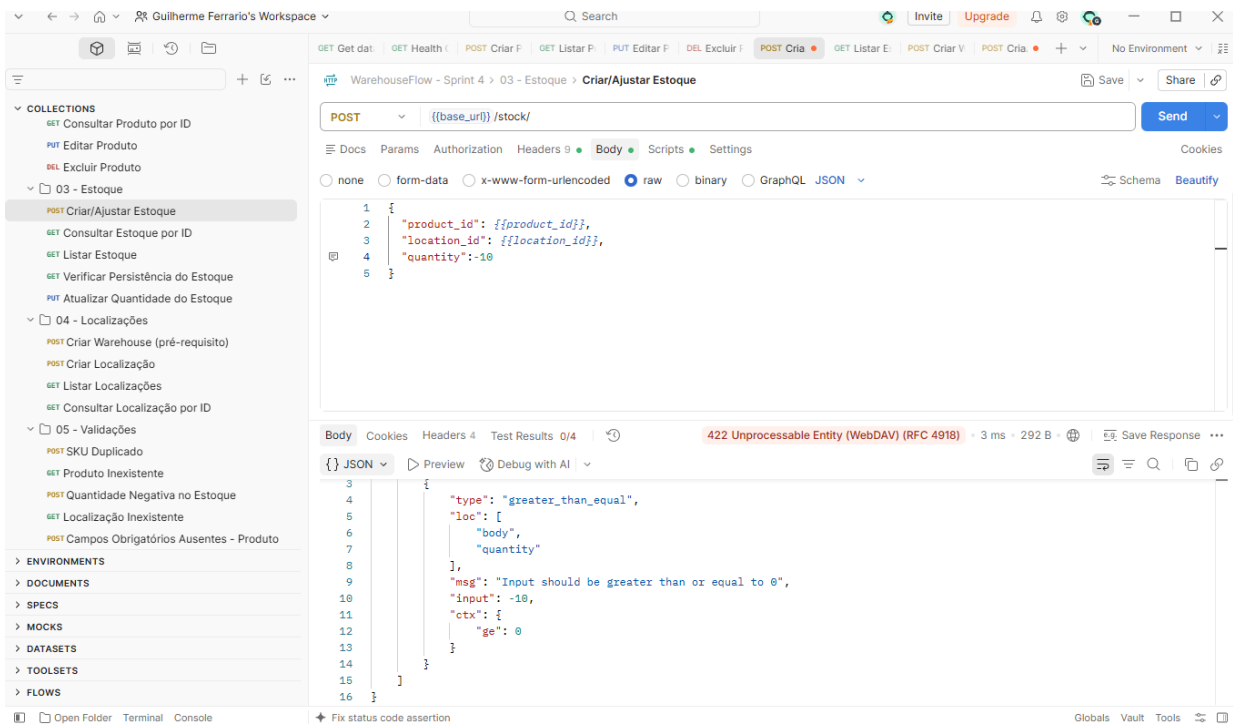
CT 03 — Exclusão de produto. Retorno 204 No Content.



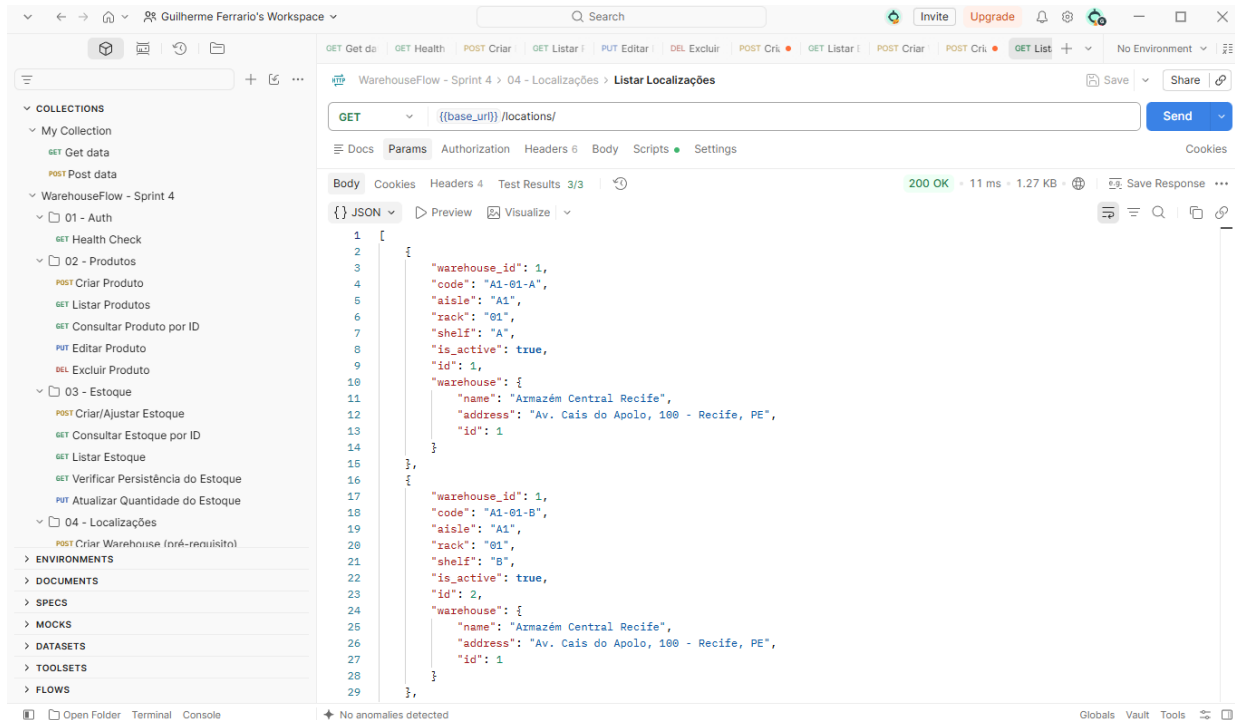
CT 04 — Criação/ajuste de estoque. Retorno 201 Created.



CT 05 — Criação de localização. Retorno 201 Created.



CT 06 — Estoque negativo rejeitado. Retorno 422 Unprocessable Entity.



CT 07 — Listagem de localizações. Retorno 200 OK.

7. Análise dos resultados

As evidências demonstram o funcionamento das operações centrais do módulo. O cadastro, a edição e a exclusão de produtos retornaram os códigos esperados. Também foi validada a criação de estoque relacionando produto, localização e quantidade, bem como o cadastro e a listagem de localizações.

No cenário de quantidade negativa, a API retornou 422 Unprocessable Entity e informou que o campo quantity deve ser maior ou igual a 0. Dessa forma, a validação de entrada foi efetivamente acionada.

8. Cenários previstos para complementação

Os seguintes pontos faziam parte do roteiro geral do Sprint 04 e não possuíam captura correspondente na primeira consolidação desta etapa. Commits, validação de erro no login, navegação entre os módulos e a persistência após reinicialização dos containers foram evidenciados na seção 10; os demais permanecem em aberto:

- Login com credenciais válidas e controle de acesso.
- SKU duplicado.
- Produto ou localização inexistente.
- Campos obrigatórios ausentes.

9. Ferramentas utilizadas

- Postman — execução dos testes da API.
- Docker / Docker Compose — execução dos serviços do projeto.
- API REST do WarehouseFlow — operações de produtos, estoque e localizações.
- PostgreSQL — persistência do sistema, conforme a arquitetura do projeto.

10. Evidências complementares da Sprint 04

Esta seção reúne evidências adicionais coletadas após a primeira consolidação da Sprint 04, complementando os cenários de teste da API (seção 6) com capturas da aplicação em execução, do repositório no GitHub e da infraestrutura de containers, incluindo uma verificação explícita de persistência de dados após a reinicialização do banco.

10.1 Histórico de commits no GitHub

O repositório do projeto reúne o histórico de commits da equipe, evidenciando a evolução do código ao longo da Sprint 04: integração de autenticação, CRUD de produtos e estoque, cadastro de posições e organização do monorepo.

Commits on Sep 25, 2026		
Organiza monorepo: unifica routers/SQL, Docker 8080/8000/5433 e CSS do login	thiagomoraishi committed 18 minutes ago	56eaf0d <>
Merge pull request #23 from thiagodemoraishi87/feature/database-crud	thiagodemoraishi87 authored 1 hour ago	Verified 8412532 <>
Merge branch 'dev' into feature/database-crud	thiagomoraishi and cursoragent committed 1 hour ago	07eaf6e <>
Merge pull request #22 from thiagodemoraishi87/feature/frontend-sprint-2-3	thiagodemoraishi87 authored 2 hours ago	Verified 461cc83 <>
Commits on Sep 24, 2026		
feat(frontend): tela de posicoes	sergiojose1 committed 12 hours ago	3dec0f2 <>
feat(frontend): integracao autenticao JWT	sergiojose1 committed 13 hours ago	49644ec <>
entradas e saidas (POST /stock/{id}/inbound e /outbound)	JoaoPedroS05 committed 13 hours ago	48355fb <>
CRUD de posições e armazéns com validações e erros 400/404/409	JoaoPedroS05 committed 13 hours ago	516a646 <>
merge: atualiza feature com dev	sergiojose1 committed 14 hours ago	6b6b49d <>
feat: correcao de erro e stock create e update	sergiojose1 committed 15 hours ago	460565e <>

Evidência 02 — Histórico de commits no repositório GitHub (commits de 25/09/2026).

Commits on Sep 22, 2026		
feat(frontend): Products CRUD	sergiojose1 committed 3 days ago	2530a74 <>
Commits on Sep 21, 2026		
feat: att login page	sergiojose1 committed 4 days ago	97c687d <>
feat: dashboard layout	sergiojose1 committed 4 days ago	53e135e <>
Merge pull request #21 from thiagodemoraishi87/feature/optimization	thiagodemoraishi87 authored 4 days ago	Verified 0c26633 <>
feat: integra auth JWT, RBAC nas rotas CRUD e login.js	thiagomoraishi committed 4 days ago	86e1d4 <>

Evidência 03 — Histórico de commits no repositório GitHub (commits de 21 a 22/09/2026).

10.2 Autenticação — validação de erro

Além dos cenários de sucesso, a tela de login foi testada com credenciais inválidas, confirmando que a aplicação impede o acesso e exibe uma mensagem de erro apropriada ao usuário.

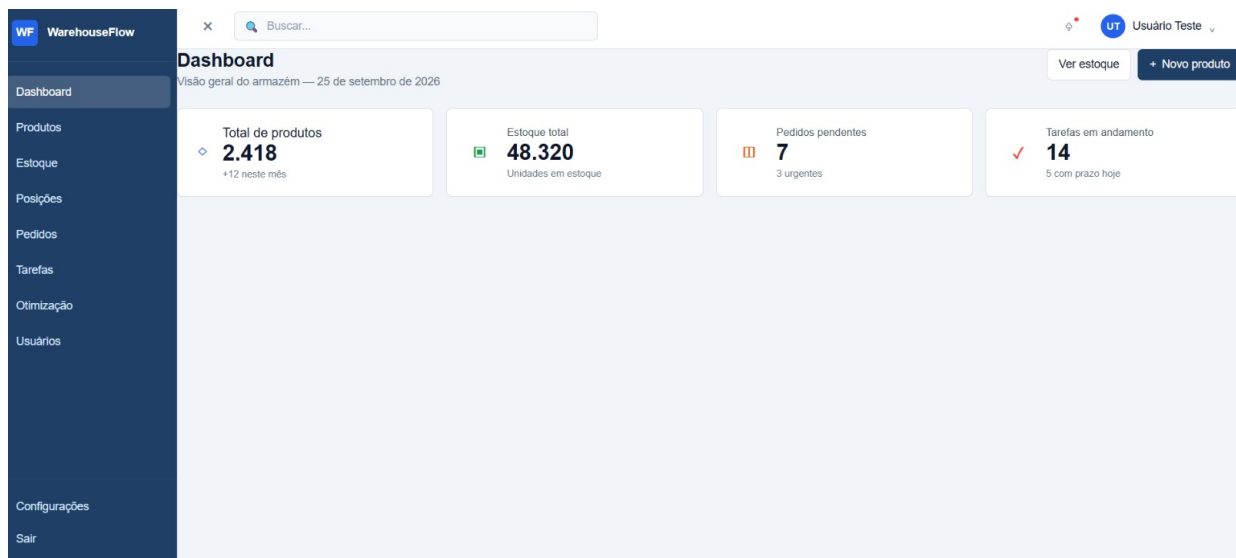


The image shows the WarehouseFlow login interface. At the top, there's a logo with a padlock icon and the text "WarehouseFlow". Below it, the heading "Bem-vindo ao WarehouseFlow" is followed by the subtitle "Gerencie seu armazém de forma simples e eficiente". The login form includes an "Email" field with the value "admin@warehouseflow.com" and a "senha" (password) field with masked characters. There are checkboxes for "Lembrar de mim" and a link for "Esqueci minha senha". A red error message "E-mail ou senha incorretos." is displayed below the password field. At the bottom is a large blue "Entrar" button.

Evidência 04 — Tentativa de login com credenciais inválidas, exibindo a mensagem "E-mail ou senha incorretos."

10.3 Navegação entre os módulos do sistema

Após autenticado, o usuário navega pelos módulos consolidados nesta Sprint: Dashboard, Produtos, Estoque e Posições, todos consultando dados reais persistidos no PostgreSQL.



The image displays the WarehouseFlow dashboard. On the left is a dark blue sidebar with a navigation menu containing: "Dashboard", "Produtos", "Estoque", "Posições", "Pedidos", "Tarefas", "Otimização", "Usuários", "Configurações", and "Sair". The main content area has a header with a search bar and a user profile "Usuário Teste". Below the header, the dashboard title "Dashboard" is followed by the subtitle "Visão geral do armazém — 25 de setembro de 2026". The dashboard features four key metrics cards: "Total de produtos" (2.418, +12 neste mês), "Estoque total" (48.320, Unidades em estoque), "Pedidos pendentes" (7, 3 urgentes), and "Tarefas em andamento" (14, 5 com prazo hoje). At the top right of the dashboard area are buttons for "Ver estoque" and "+ Novo produto".

Evidência 05 — Dashboard: visão geral do armazém.

WF WarehouseFlow

Dashboard

Produtos

Estoque

Posições

Pedidos

Tarefas

Otimização

Usuários

X

Buscar...

UT Usuário Teste

Produtos

Gerencie os produtos cadastrados no armazém.

+ Novo produto

Buscar por nome ou SKU...

SKU	NOME	DESCRIÇÃO	PESO	VOLUME	AÇÕES
PROD-001	Monitor LED 24"	Monitor FHD 1080p para estações de trabalho	3.50 kg	0.03 m³	EditarExcluir
PROD-002	Teclado Mecânico RGB	Teclado mecânico switch blue N-Key Rollover	0.95 kg	0.01 m³	EditarExcluir
PROD-003	Cabo HDMI 2.0 2m	Cabo HDMI alta velocidade banhado a ouro	0.15 kg	0.00 m³	EditarExcluir
PROD-004	Gancho	—	5.00 kg	10.00 m³	EditarExcluir

Evidência 06 — Módulo Produtos: listagem cadastrada no sistema.

WF WarehouseFlow

Dashboard

Produtos

Estoque

Posições

Pedidos

Tarefas

Otimização

Usuários

Configurações

Sair

X

Buscar...

UT Usuário Teste

Estoque

Consulte e gerencie o saldo de produtos por posição.

+ Ajustar estoque

Buscar por produto, SKU ou posição...

SKU	PRODUTO	POSIÇÃO	QUANTIDADE	ÚLTIMA ATUALIZAÇÃO	AÇÕES
PROD-001	Monitor LED 24"	A1-01-A	15	25/09/2026, 13:32:25	Ajustar
PROD-002	Teclado Mecânico RGB	A1-01-B	40	25/09/2026, 13:32:25	Ajustar
PROD-003	Cabo HDMI 2.0 2m	A1-02-A	100	25/09/2026, 13:32:25	Ajustar
PROD-004	Gancho	B1-01-A	4	25/09/2026, 13:44:21	Ajustar

Evidência 07 — Módulo Estoque: saldo por posição.

WF WarehouseFlow

Dashboard

Produtos

Estoque

Posições

Pedidos

Tarefas

Otimização

Usuários

Configurações

Buscar...

UT Usuário Teste

Posições

Gerencie as posições de armazenamento do armazém.

+ Nova posição

Buscar por código, corredor, rack... Todos os armazéns

Código	Armazém	Corredor	Rack	Prateleira	Status	Ações
A1-01-A	Armazém Central Recife	A1	01	A	Ativa	Editar Excluir
A1-01-B	Armazém Central Recife	A1	01	B	Ativa	Editar Excluir
A1-02-A	Armazém Central Recife	A1	02	A	Ativa	Editar Excluir
B1-01-A	Armazém Central Recife	B1	01	A	Ativa	Editar Excluir
B1-01-B	Armazém Central Recife	B1	01	B	Ativa	Editar Excluir

Evidência 08 — Módulo Posições: localizações físicas do armazém.

10.4 Persistência de dados após reinicialização dos containers

Para fechar o cenário de persistência previsto na seção 8, o container do banco de dados (warehouseflow-db) foi reiniciado e os dados foram conferidos em seguida. O log abaixo mostra o desligamento (fast shutdown) e a nova inicialização do PostgreSQL, seguido da confirmação de que os quatro serviços do projeto (warehouseflow, db, api e frontend) permanecem em execução.

warehouseflow-db

cf5e5665e6a3 postgres:16-alpine 5433:5432

STATUS Running (0 seconds ago)

Logs Inspect Bind mounts Exec Files Stats

2026-09-25 14:48:26.688 UTC [1] LOG: starting PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit

2026-09-25 14:48:26.690 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432

2026-09-25 14:48:26.690 UTC [1] LOG: listening on IPv6 address ":::", port 5432

2026-09-25 14:48:26.696 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"

2026-09-25 14:48:26.726 UTC [30] LOG: database system was shut down at 2026-09-25 14:48:25 UTC

2026-09-25 14:48:26.754 UTC [1] LOG: database system is ready to accept connections

2026-09-25 14:50:05.201 UTC [1] LOG: received fast shutdown request

2026-09-25 14:50:05.204 UTC [1] LOG: aborting any active transactions

2026-09-25 14:50:05.207 UTC [1] LOG: background worker "logical replication launcher" (PID 33) exited with exit code 1

2026-09-25 14:50:05.211 UTC [28] LOG: shutting down

2026-09-25 14:50:05.214 UTC [28] LOG: checkpoint starting: shutdown immediate

2026-09-25 14:50:05.230 UTC [28] LOG: checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.005 s, sync=0.003 s, total=0.019 s; sync files=2, longest=0.002 s, average=0.002 s; distance=0 kB, estimate=0 kB; lsn=0/19A48B8, redo lsn=0/19A48B8

2026-09-25 14:50:05.238 UTC [1] LOG: database system is shut down

PostgreSQL Database directory appears to contain a database; Skipping initialization

2026-09-25 14:50:05.905 UTC [1] LOG: starting PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit

2026-09-25 14:50:05.906 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432

2026-09-25 14:50:05.906 UTC [1] LOG: listening on IPv6 address ":::", port 5432

2026-09-25 14:50:05.912 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"

2026-09-25 14:50:05.920 UTC [30] LOG: database system was shut down at 2026-09-25 14:50:05 UTC

2026-09-25 14:50:05.930 UTC [1] LOG: database system is ready to accept connections

Evidência 09 — Log do container warehouseflow-db: shutdown e reinicialização do PostgreSQL.

☐	warehouseflow	-	-	-	5.38%	143.74MB / 11.1i	3.8%	16.4KB /				
☐	db	cf5e5665e6a3	postgres:1	5433:5432	3.22%	40.36MB / 3.7GB	1.07%	16.4KB /				
☐	api	09461b5d9c9f	warehouse	8000:8000	2.16%	95.74MB / 3.7GB	2.53%	0B / 0B				
☐	frontend	abae4118fef7	nginx:alpin	8080:80	0%	7.64MB / 3.7GB	0.2%	0B / 4.1i				

Evidência 10 — Containers do WarehouseFlow (warehouseflow, db, api, frontend) em execução após o restart do banco.

Em seguida, o módulo Estoque foi consultado novamente: os saldos e os horários de "última atualização" (13:32:25 e 13:44:21) permaneceram idênticos aos registrados antes da reinicialização (log às 14:50), confirmando que os dados persistiram corretamente.

WF WarehouseFlow	X	Buscar...	UT Usuário Teste
Dashboard	Estoque		
Produtos	Consulte e gerencie o saldo de produtos por posição.		
Estoque	Buscar por produto, SKU ou posição...		
Posições	SKU	PRODUTO	POSICÃO
Pedidos	PROD-001	Monitor LED 24"	A1-01-A
Tarefas	PROD-002	Teclado Mecânico RGB	A1-01-B
Otimização	PROD-003	Cabo HDMI 2.0 2m	A1-02-A
Usuários	PROD-004	Gancho	B1-01-A
Configurações			
Sair			

Evidência 11 — Módulo Estoque consultado após a reinicialização do container do banco, com saldos e timestamps preservados — persistência confirmada.

11. Repositório e protótipo

GitHub: <https://github.com/thiagodemorais87/WarehouseFlow>

Figma: <https://www.figma.com/design/uGceagehdqafcFckYSIWZY/Sem-título?node-id=5-569&t=LlfUx9e9ylwhR2CV-0>

12. Dificuldades encontradas

Foi necessário atualizar o WSL para permitir a execução adequada do Docker no ambiente local. Após a atualização, os containers do projeto puderam ser executados e os testes da API foram realizados pelo Postman.

13. Próximos passos

- Executar e registrar os cenários ainda não evidenciados (SKU duplicado, produto/localização inexistente, campos obrigatórios ausentes), caso sejam exigidos.
- Consolidar os Sprints 01 a 04 em um único PDF para a entrega.
- Prosseguir para as funcionalidades dos próximos Sprints.

14. Considerações finais

O Sprint 04 apresentou evidências funcionais das operações de Produtos, Estoque e Localizações. Os resultados registrados no Postman demonstram respostas HTTP compatíveis com as operações executadas e uma validação explícita para impedir quantidade negativa no estoque. As evidências complementares desta revisão — commits no GitHub, validação de erro no login, navegação entre os módulos consolidados e a verificação de persistência após a reinicialização dos containers — fecham os principais pontos em aberto identificados anteriormente.